

ParaDiGM: Parallel Distributed General Mesh

Eric Quémerais¹, Bastien Andrieu¹, Bruno Maugars², Karmijn Hoogveld¹, Clément Benazet², Julien Coulet², Berenger Berthoul², Nicolas Dellinger³, and Thomas Hennion¹

¹ DMPE, ONERA, Université Paris-Saclay, 92320 Châtillon, France ² DAAA, ONERA, Université Paris-Saclay, 92320 Châtillon, France ³ DMPE, ONERA, Université de Toulouse, 31000 Toulouse, France

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

ParaDiGM: Parallel Distributed General Mesh

Summary

ParaDiGM (Parallel Distributed General Mesh) is an open-source C library (LGPL) designed to overcome the technological bottlenecks associated with geometric data handling and mesh manipulation in massively parallel numerical simulations. While historically branched from the **CWIPI** (Quémerais et al., 2026) coupling library, ParaDiGM has evolved into a standalone infrastructure upon which **CWIPI** now relies for all its distributed geometric computations.

As a high-performance middleware, ParaDiGM provides essential geometric services—such as parallel wall distance computation, point cloud location, and dynamic repartitioning—to various numerical simulation and pre-processing software, including **CEDRE** (Refloch et al., 2011), **elsA** (Cambier et al., 2013), **SoNiCS** (Lienhardt & Michel, 2025), **MoDeTheC** (Dellinger et al., 2024) and **Maia** (Coulet, 2026). By generalizing the **Partitioned** and **Distributed View** concepts, it ensures that every stage of the computation remains memory-balanced, even for meshes exceeding 100 billion elements. With native APIs for **C/C++**, **Fortran**, and **Python/NumPy**, ParaDiGM serves as a versatile infrastructure for aerospace research and large-scale computational physics in the exascale era.

Statement of Need

In the era of exascale computing, numerical simulation faces a critical shift. In **Computational Fluid Dynamics (CFD)**, mesh management can no longer rely on centralized or semi-distributed approaches. Integrating distributed mesh capabilities into existing production solvers often encounters major architectural and performance barriers.

From Static to Dynamic and Repetitive Geometry

Geometric algorithms, once confined to pre-processing, are now required repeatedly within the solver's main execution loop. Modern studies involve evolving meshes, driven either by **moving bodies** or **Dynamic Mesh Adaptation**. Whether computing wall distances for turbulence modeling or performing mesh repartitioning, these operations must deliver execution times of the same order of magnitude as a solver iteration. Failing to meet this constraint creates a performance bottleneck that severely penalizes the overall simulation efficiency.

The Challenge of Dynamic Load Balancing

A further bottleneck lies in data distribution: the initial partition provided to geometric algorithms is typically optimized for physical computation, which is often **sub-optimal for geometric operations**. Without internal dynamic load-balancing, these operations create

39 computation imbalances and memory bottlenecks, compromising the execution of simulations
40 at a very large scale.

41 Limitations of Current Solutions

42 Existing geometric frameworks frequently impose constraints that hinder their adoption:

- 43 ■ **Rigid Data Structures:** Unlike heavy frameworks (Trilinos (Heroux et al., 2005), Arcane
44 (GrosPELLIER & Lelandais, 2009)), ParaDiGM does not enforce complex object hierarchies.
45 This **non-intrusive** “Progressive Framework” approach allows adoption without a deep
46 refactoring of the host solver.
- 47 ■ **Monolithic Data Models:** Unlike platforms such as Salome (MED format) (Ribes &
48 Caremoli, 2007), ParaDiGM relies on simple **CSR arrays**, avoiding software overhead
49 and memory peaks critical for optimized solvers.

50 The ParaDiGM Approach: A Progressive Middleware

51 ParaDiGM is a **Software Development Kit (SDK)** rather than a standalone application. Unlike
52 tools like Gmsh (Geuzaine & Remacle, 2009), it is not intended for mesh generation from
53 CAD models. Its role begins as soon as a **first mesh is obtained**: it provides solvers with
54 the low-level functions needed to distribute, partition, and manipulate discretized meshes in a
55 parallel fashion.

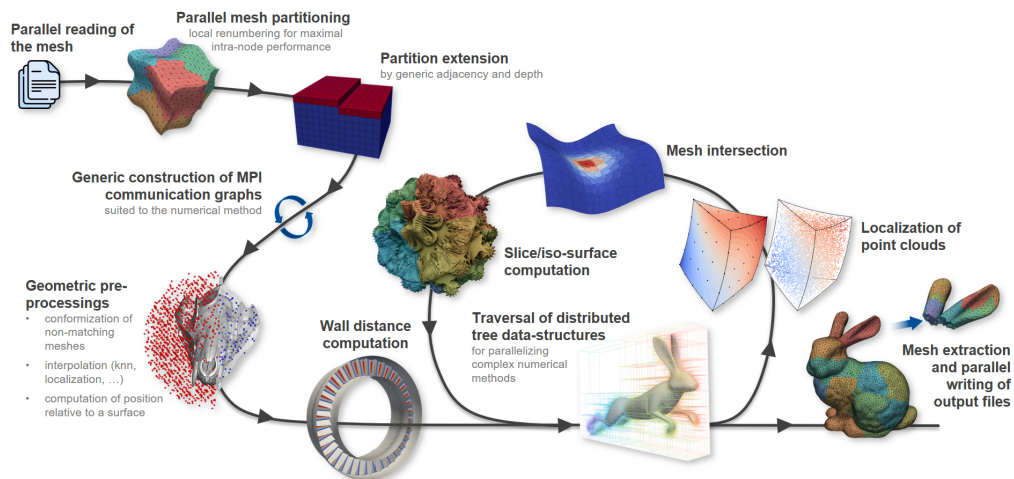


Figure 1: Example of a simulation chain using the features offered by ParaDiGM.

56 Its philosophy is built on three pillars:

- 57 1. **Interoperability and Agnosticism:** Ensuring full compatibility with legacy languages
58 (Fortran, C) and modern environments (Python/NumPy) for a wide variety of numerical
59 methods (Finite Volumes, Finite Elements, SPH).
- 60 2. **Hybrid Partitioning Strategy:** ParaDiGM unifies third-party solutions (e.g., PT-Scotch
61 (Chevalier & Pellegrini, 2008), ParMetis (Karypis et al., 1997)) while supplementing them
62 with native high-performance algorithms like **Space Filling Curves (SFC)** for frequent
63 repartitioning.
- 64 3. **Total Distribution and Geometric Performance:** By ensuring homogeneous load
65 distribution, the framework has enabled the development of highly efficient algorithms,
66 such as **distributed point cloud location** (Andrieu et al., 2026), achieving performance
67 levels compatible with solver iteration frequencies.

Ongoing Work and Future Challenges

A major challenge for modern numerical simulation is the transition from CPU-based algorithms to implementations optimized for **GPU architectures**. To this end, recent work has been conducted to port ParaDiGM's most critical components to these accelerators (Cazalbou et al., 2024).

These developments, intended to be industrialized and integrated into a future version, focus on:

- **Hybrid Parallel Octrees:** The construction and traversal of search trees (octrees) are generally the most memory-intensive and time-consuming steps in geometric algorithms.
- **CPU/GPU Optimization:** By offloading these massive data structures to GPUs, ParaDiGM aims to drastically reduce turnaround times for location operations while maintaining the load balance essential for exascale performance.

This evolution will ensure that ParaDiGM remains a state-of-the-art infrastructure, capable of taking full advantage of the next generation of heterogeneous supercomputers.

Acknowledgements

The authors would like to thank the following people for their contributions to the development, testing, and dissemination of the ParaDiGM library within the community:

Sébastien Bourasseau¹, Nicolas Lantos¹, Niels Guilbert¹, Alain Hervault¹, Julien Magnenet¹, Lucas Manueco¹, Lionel Matuszewski¹, Yacine Mezemate¹, Bertrand Michel¹, Christina Paulin¹, Christophe Peyret¹, Julien Vanharen¹, Jérôme Esbrat², Victor Pacotte², Bruno Peres² and Mickael Philit³.

¹ ONERA ² EOLEN ³ Safran

The authors also wish to thank BPI France, the Directorate General for Civil Aviation (DGAC), and the General Scientific Directorate of ONERA for their financial support.

Author contributions

The contributions to this software are listed according to the CRediT taxonomy:

- **Eric Quémerais:** Conceptualization, Methodology, Software, Validation, Writing – Review & Editing, Project Administration, Funding Acquisition, Supervision
- **Bastien Andrieu:** Software, Validation, Writing – Original Draft
- **Bruno Maugars:** Software, Validation
- **Karmijn Hoogveld:** Software, Validation, Writing – Original Draft
- **Clément Benazet:** Software, Validation
- **Juïen Coulet:** Software, Validation
- **Bérenger Berthoul:** Software
- **Nicolas Dellinger:** Software, Validation
- **Thomas Hennion:** Software, Validation

References

- Andrieu, B., Maugars, B., & Quémerais, E. (2026). Dynamic load-balanced point location algorithm for data mapping. *Comptes Rendus. Mécanique*, 354(G1), 53–70. <https://doi.org/10.5802/crmeca.335>

- 108 Cambier, L., Heib, S., & Plot, S. (2013). The onera elsA CFD software: Input from research
109 and feedback from industry. *Mechanics & Industry*, 14, 159–174. [https://doi.org/10.1051/
110 meca/2013056](https://doi.org/10.1051/meca/2013056)
- 111 Cazalbou, R., Duchaine, F., Quémerais, E., Andrieu, B., Staffebach, G., & Maugars, B. (2024).
112 *Hybrid multi-GPU distributed octrees construction for massively parallel code coupling*
113 *applications*. <https://doi.org/10.1145/3659914.3659928>
- 114 Chevalier, C., & Pellegrini, F. (2008). PT-scotch: A tool for efficient parallel graph ordering.
115 *Parallel Computing*, 34(6), 318–331. <https://doi.org/10.1016/j.parco.2007.12.001>
- 116 Coulet, J. (2026). *Maia*. <https://github.com/onera/maia>.
- 117 Dellinger, N., Leplat, G., Huchette, C., Biasi, V., & Feyel, F. (2024). Numerical modeling and
118 experimental validation of heat and mass transfer within decomposing carbon fibers/epoxy
119 resin composite laminates. *International Journal of Thermal Sciences*, 201, 109040.
120 <https://doi.org/10.1016/j.ijthermalsci.2024.109040>
- 121 Geuzaine, C., & Remacle, J.-F. (2009). Gmsh: A 3-d finite element mesh generator with
122 built-in pre- and post-processing facilities. *International Journal for Numerical Methods in*
123 *Engineering*, 79(11), 1309–1331. <https://doi.org/10.1002/nme.2579>
- 124 Gropellier, G., & Lelandais, B. (2009). *The arcane development framework*. [https://doi.org/
125 10.1145/1595655.1595659](https://doi.org/10.1145/1595655.1595659)
- 126 Heroux, M., Bartlett, V., R.and Howle, Hoekstra, R., Hu, J., Kolda, T., Lehoucq, R.,
127 Long, K., Pawlowski, R., Phipps, E., & others. (2005). An overview of the trilinos
128 project. *ACM Transactions on Mathematical Software (TOMS)*, 31(3), 397–423. <https://doi.org/10.1145/1089014.1089021>
- 129 Karypis, G., Schloegel, K., & Kumar, V. (1997). *PARMETIS: Parallel graph partitioning and*
130 *sparse matrix ordering library* (No. 97-060). University of Minnesota. [https://hdl.handle.
131 net/11299/215345](https://hdl.handle.net/11299/215345)
- 132 Lienhardt, M., & Michel, B. (2025). SoNICS: Design and workflow of a new CFD software.
133 *Comptes Rendus. Mécanique*, 353, 1261–1287. <https://doi.org/10.5802/crmeca.334>
- 134 Quémerais, E., Andrieu, B., Maugars, B., & Hoogveld, K. (2026). *CWIPI*. [https://github.
135 com/onera/cwipi](https://github.com/onera/cwipi).
- 136 Refloch, A., Courbet, B., Murrone, A., Villedieu, P., Laurent, C., Gilbank, P., Troyes, J., Tessé,
137 L., Chaineray, G., Dargaud, J. B., Quémerais, E., & Vuillot, F. (2011). CEDRE Software.
138 *Aerospace Lab*, 2, p. 1–10. <https://hal.science/hal-01182463>
- 139 Ribes, A., & Caremoli, C. (2007). Salomé platform component model for numerical simulation.
140 *31st Annual International Computer Software and Applications Conference (COMPSAC*
141 *2007)*, 2, 553–564. <https://doi.org/10.1109/COMPSAC.2007.185>